

Challenge

Create an e-commerce web app for displaying products, categories, and product details.

The app will also let the user authenticate and access protected features like Shopping Cart and Wishlist.

We didn't provide any mockup or design to let you express your creativity, just make sure to respect every Acceptance Criteria and basic principles of usability.

The goal is to evaluate how you organize your code, how you interact with APIs, and your attention to detail.

Required Tech Stack

1. TypeScript is mandatory.
2. For the UI, you must use a modern framework (e.g., React, Vue, or Angular). Vanilla JavaScript is not permitted for this challenge to ensure a standardized evaluation of component-driven architectures.
3. Fake Store API (<https://fakestoreapi.com/docs>) as the backend API to fulfill the requirements.

You are free to use extra Javascript libraries to achieve the goal, as long as they are compatible with the requirements listed above.

You can use CSS frameworks or utility libraries (like Tailwind CSS) if you wish, but writing your own custom styles (using CSS Modules, SCSS, or Styled Components) is strongly preferred to demonstrate your understanding of CSS architecture.

Acceptance Criteria

1. Repository and Workflow

- a. The project must be hosted in a public GitHub repository.
- b. Use clean commit messages, producing clean commit histories.
- c. The repository must include a readme.md in the root with precise local setup instructions.
- d. If you use AI tools, please include a brief paragraph in your readme.md explaining how you used them (e.g., for scaffolding, debugging, regex generation, or refactoring) and how you validate the generated output.

2. Layout and Responsiveness and Accessibility (a11y)

- a. The global layout must consist of a Header and a Main area to display the views.
- b. The web app must be responsive and fully usable on tablets and smartphones.
- c. Use semantic HTML5 markup, ensure basic keyboard navigation, and include appropriate `alt` attributes for all product images.

3. Header

- a. It must display the store name, a logo (textual is fine), and the category navigation.
- b. Categories must be fetched dynamically via the backend API.

4. Home (view)

- a. It must display a grid of product cards fetched via the API. Each card must display at least the product image, name, and price.
- b. Each product card must be clickable; upon clicking, the user navigates to the product detail view.
- c. By clicking on a category in the Header, the user must be able to filter the products on the Home page for that specific category (using the API).
- d. When a category filter is applied, the URL (Query String or Path) must update accordingly. Furthermore, if a user directly accesses a filtered URL (e.g., via deep link or page refresh), the application must correctly read the URL and initialize the interface with the appropriate filter applied.

5. Product (view)

- a. By clicking on a product, the user navigates to a detail view displaying the product's name, image, price, and full description.

Bonus points

The following points are not mandatory, but completing one or more of them will be highly appreciated during the evaluation process, as they reflect the "Preferred Qualifications" and values of our Job Description.

1. Repository and Workflow

- a. Use the Pull Request system to merge separate branches to main, with clear PR Description.
- b. Instead of committing everything to the main branch, simulate a real workflow: develop a feature in a separate branch and open a Pull Request to the main branch on GitHub.

2. Tech capabilities

- a. Develop the application using the framework: Vue.js.
- b. Strict use of TypeScript to strongly type API responses and data structures.
- c. Use of SCSS or CSS variables (Custom Properties) to simulate a Design System-based approach (e.g., centralized management of primary colors, spacing, and typography).
- d. Writing 1 or 2 unit tests (e.g., using Jest, Vitest, or your framework's native testing library) for a critical utility function or UI component.
- e. Implement State Management logics

3. Visuals and Functionalities

- a. The app looks clean, polished, and handles smooth transitions or animations when navigating between views.
- b. Visual management of loading states (e.g., spinners or skeleton loaders during API calls)
- c. Visual management of network errors (e.g., modal window with the classic "something went wrong" message).
- d. The web app supports switching between light and dark themes.
- e. Add a simple "Add to cart" button on the detail page, which increments a counter visible in the Header (even just via local client state, without complex persistence).
- f. Integrate the Login and Logout feature using the dedicated Backend API.